DAVAO ORIENTAL STATE UNIVERSITY
FACULTY OF COMPUTING, ENGINEERING AND TECHNOLOGY

LABORATORY EXERCISE

ITP 141 · Systems Administration and Maintenance

BS Information Technology Program · Mati City, Davao Oriental · Philippines



Course Code / Title:	ITP 141 · Systems Administration and Maintenance
Program:	BS Information Technology
Laboratory Exercise:	Lab 1A: Bash Scripting (Basic to Intermediate), Six-Script Suite
Scheduled Week:	Week 6–7
Semester:	First Semester, AY 2026–2027
Instructor:	Ar-Jay R. Sacay, MIT

Lab 1A: Bash Scripting (Basic to Intermediate), Six-Script Suite

Total Points: 50 pts | CLOs: CLO1, CLO2 | Module 1 | Week 6–7

Module / Topic:	Module 1, Topic 5: Intermediate Bash Scripting
Tool / Platform:	Ubuntu Server 24.04 LTS (ITSU laboratory environment or WSL 2) | Bash 5.2 or later | nano / vim
Group Composition:	Group of 4

★ SCENARIO (DOrSU, ITSU and the BSIT Program)

The Information Technology Services Unit (ITSU) of Davao Oriental State University maintains a small group of Ubuntu servers that carry the BSIT Program's internal file shares, the student attendance database, and several faculty project sites. Because only two technicians support the whole program, routine housekeeping is done by hand: oversized log files are rotated, the shared drive is copied to a second disk, and the accounts consuming the most storage are reviewed. During enrolment these tasks are quietly skipped, which is precisely when the servers are under the heaviest load. ITSU has asked your group to remove this work from the technicians by writing a suite of Bash scripts that run without supervision. Build the six scripts required by this exercise so that the BSIT Program server can rotate its own logs, produce timestamped rsync backups, and report disk usage automatically on a cron schedule.

Learning Objectives

1. Write six Bash scripts (four numbered task scripts together with log_rotate.sh and backup_auto.sh) that apply variables, command substitution, ANSI colour output, conditionals with test operators, loops, functions with positional arguments, arrays, and parameter expansion.
2. Implement log_rotate.sh, using a function that takes the log path as a positional argument, so that exactly the five most recent versions of a log file are retained.
3. Build backup_auto.sh with strict mode (set -euo pipefail), a trap on ERR, rsync, timestamped destination directories, and an append-only log file, then verify the backup by restoring one file from it.
4. Schedule script2_diskcheck.sh with cron and verify execution from both the redirected log file and the cron service journal.

Materials / Tools Required

• Ubuntu Server 24.04 LTS with sudo access: the ITSU laboratory environment used for Topics 1 to 5, or a WSL 2 distribution if your group has already enabled one. The VirtualBox guests are not built until Laboratory Exercises 1B and 1C, so this exercise does not require them. Bash 5.2 or later is already installed.
• Text editor: nano or vim
• References: Robbins, A., & Beebe, N. H. F. (2022). Classic shell scripting (2nd ed.). O'Reilly Media. Nugraha, L. (2024, June 25). Bash scripting tutorial for beginners. Hostinger Tutorials.
• Lab worksheet (this document)

Group Member Roles

Member / Role	Primary Responsibility
JEREMIAH JALANDOON (Script Writer A)	Writes script1_sysinfo.sh (variables, command substitution, ANSI colour) and script2_diskcheck.sh (if-elif-else threshold logic).
KRISTY ESTILLORE (Script Writer B)	Writes script3_userreport.sh (array and check_user function) and script4_cleanup.sh (strict mode, trap, and exit codes).
DAISY GRACE GENERALE (Log Rotation & Backup Dev)	Writes log_rotate.sh and backup_auto.sh, using rsync, parameter expansion, and trap.
CYRINE GANLOY (Scheduler & Tester)	Configures cron, runs all six scripts, completes the execution table, and compiles the group report.

Step-by-Step Procedure

Working note: the six scripts are specified by what they must do, not line by line. If a command behaves differently on your Bash version, or a test file is not where a step expects it, adapt the script so that the required behaviour still holds, and record the change in the script header comments.

Step	Instruction
1	Create the working directory: mkdir ~/lab1a_bash && cd ~/lab1a_bash. All six scripts are written in this directory.
2	Member 1, script1_sysinfo.sh: Store the hostname, uptime, current user, date, and kernel version in variables, obtaining each through command substitution. Print every value in ANSI colour (echo -e "\e[32m$var\e[0m"). Test with bash script1_sysinfo.sh and capture the output.
3	Member 1, script2_diskcheck.sh: Read the usage of / with df. Test the thresholds in descending order, since an ascending test can never reach the higher branch: print CRITICAL above 90 per cent, WARNING above 70 per cent, and OK otherwise. Use if-elif-else with integer comparison, and test the script against all three outcomes.
4	Member 2, script3_userreport.sh: Declare USERS=(sysadmin root nobody). Write a function check_user() that takes the account name as its positional argument $1 and prints either the output of id or NOT FOUND. Loop over the array calling the function for each element, and report the element count with ${#USERS[@]}.
5	Member 2, script4_cleanup.sh: Begin with set -euo pipefail, then register trap 'echo "ERROR at line $LINENO" >> ~/lab1a_bash/error.log; exit 1' ERR. Create a disposable test tree first (mkdir -p ~/lab1a_bash/tmptest, then create files and age them with touch -d '10 days ago'), and run find ~/lab1a_bash/tmptest -mtime +7 -delete against it. Never point this command at the live /tmp, because files still in use by running services would be deleted. Exit 0 on success. Trigger an intentional error and confirm that error.log records it.

6	Member 3, log_rotate.sh: Write rotate_log(), taking the log path as $1. Delete version .5, then shift .4 to .5, .3 to .4, .2 to .3, .1 to .2, and the current file to .1. Build the numbered names with parameter expansion, for example ${1%.log}, rather than hard-coding them. Create ~/lab1a_bash/logs/app.log and append a line to it before each rotation. Run rotate_log seven times, listing the directory after each run, and confirm that the numbered versions never exceed five (.1 to .5) and that the oldest is discarded on every run after the fifth.
7	Member 3, backup_auto.sh: Begin with set -euo pipefail. Set DEST=~/lab1a_bash/backup_$(date +%Y%m%d_%H%M%S) and run mkdir -p "$DEST", then rsync -avz ~/lab1a_bash/logs/ "$DEST"/ >> backup.log 2>&1. Register trap 'echo "FAILED $(date -Is)" >> backup.log; exit 1' ERR, and echo the start and end timestamps. Run the script twice and confirm two separate timestamped directories. Finally restore one file from the first directory, since a backup that has never been restored is an assumption rather than a backup.
8	Member 4, cron: run crontab -e and add the entry below, using absolute paths because the tilde is not reliably expanded by every cron implementation: */5 * * * * /bin/bash /home/sysadmin/lab1a_bash/script2_diskcheck.sh >> /home/sysadmin/lab1a_bash/cron_disk.log 2>&1. Do not change the system clock, because cron reads it once a minute and a forward jump can skip the job entirely. Either wait for two runs (ten minutes) or set the schedule to * * * * * temporarily, confirm that two entries appear, then restore */5. Verify that cron_disk.log holds entries and cross-check the cron service log with journalctl -u cron --since "20 min ago". Use journalctl rather than grep CRON /var/log/syslog, because the minimised Ubuntu Server installation used in this course does not include rsyslog, so /var/log/syslog may not exist.
9	Member 4: run all six scripts in sequence and complete the Script Execution Table in Section 3 of the worksheet.
10	All members: complete the Bash and Python comparison table (five criteria) in Section 4. This is an analytical comparison only; no Python code is written or submitted in ITP 141.
11	All members: write a group reflection of at least 100 words on which of these housekeeping tasks you would automate first on an ITSU production server, and on the reasons for that choice.
12	Member 4: compile and submit all six .sh files, the cron_disk.log screenshot, the completed execution table, and the worksheet as a PDF, through the class LMS.

Grading Rubric (Total: 50 Points) | 4 Criteria

Criterion	Pts	What Is Being Assessed	Performance Levels
Scripts 1 & 2: Variables, ANSI Color & Conditionals	**20**	Script 1 stores all five system values in variables and prints them with correct ANSI colour output; Script 2 applies if-elif-else with the three threshold comparisons in the correct descending order.	Excellent (17–20): Both scripts run without errors; ANSI colour output present; all thresholds correct. Proficient (13–16): Both scripts run; one feature missing, for example colour absent or one threshold out of order. Developing (8–12): One script correct; other has syntax errors. Beginning (0–7): Both scripts non-functional.
Scripts 3 & 4: Function + Array & trap Error Handling	**14**	Script 3 uses an array and a check_user() function with a positional argument; Script 4 uses strict mode and a trap that logs to error.log, and returns correct exit codes.	Excellent (11–14): Both correct; function takes a positional argument; strict mode present; trap logs to file; exit codes correct. Proficient (8–10): Both run; trap present but without file logging; exit codes basic; strict mode missing. Developing (5–7): One script functional; function or trap missing. Beginning (0–4): Neither functional.
Scripts 5 & 6: Log Rotation + rsync Backup Automation	**10**	rotate_log() retains exactly five log versions and uses parameter expansion; backup_auto.sh creates two timestamped rsync directories with strict mode, trap, log entries, and a verified restore.	Excellent (9–10): Exactly five versions retained; two timestamped backup directories; trap, log entries and restore all correct. Proficient (6–8): Rotation mostly correct; backup runs but trap or restore missing. Developing (3–5): One of the two scripts functional. Beginning (0–2): Neither script functional.
Cron Schedule, Execution Table & Reflection	**6**	Cron job verified from both the redirected log and the cron journal; execution table complete for all six scripts with expected against actual output; reflection of at least 100 words.	Excellent (5–6): Cron verified in both places; table complete; reflection of 100 words or more. Proficient (3–4): Cron configured; log not verified; table mostly complete. Developing (1–2): Cron not set up; table partial. Beginning (0): Cron and table absent.
TOTAL	**50**	Submit: all required files + screenshots via class LMS	Complete the Student Group Worksheet below before submission

★ LAB 1A WORKSHEET | STUDENT GROUP GUIDE

Bash Scripting (Basic to Intermediate), Six-Script Suite | 50 pts | CLO1, CLO2

SECTION 1: LAB IDENTIFICATION

Laboratory Exercise No. & Title:	Lab 1A: Bash Scripting (Basic to Intermediate), Six-Script Suite
Module & Topic:	Module 1, Topic 5: Intermediate Bash Scripting
Group Number:	GROUP NAMEN
Date Performed:	Date Submitted:

SECTION 2: GROUP MEMBERS AND ROLE ASSIGNMENTS

No.	Full Name	Assigned Role	Tasks Completed
1	Jeremiah v. Jalando-on	**Scripts 1 & 2: Variables, ANSI Color & Conditionals**	Writes script1_sysinfo.sh (variables, command substitution, ANSI colour) and script2_diskcheck.sh (if-elif-else threshold logic).
2	Kristy L. Estillore	**Scripts 3 & 4: Function + Array & trap Error Handling**	Writes script3_userreport.sh (array and check_user function) and script4_cleanup.sh (strict mode, trap, and exit codes).
3	Daisy Grace m. Generale	**Scripts 5 & 6: Log Rotation + rsync Backup Automation**	Writes log_rotate.sh and backup_auto.sh, using rsync, parameter expansion, and trap.
4	Cyrine Ganloy	**Cron Schedule, Execution Table & Reflection**	Configures cron, runs all six scripts, completes the execution table, and compiles the group report.

SECTION 3: SCRIPT EXECUTION TABLE

Script File	Command Run	Expected Output	Actual Output (describe)	Passed? (Y/N)
script1_sysinfo.sh	bash script1_sysinfo.sh	Display hostname, uptime, current user, date, and kernel version using variables and ANSI colors	Displayed the hostname, uptime, current user, date, and kernel version successfully in colored text.	Y
script2_diskcheck.sh	bash script2_diskcheck.sh	Display current disk usage and corresponding status	Displayed OK because the root filesystem usage was below 70%.	Y

script3_userreport.sh	bash script3_userreport.sh	Check users sysadmin, root, and nobody and display total users checked	sysadmin was not found while root and nobody were found and displays their user id.	Y
script4_cleanup.sh	bash script4_cleanup.sh	Create test files, remove files older than 7 days, and display successful cleanup	Old file1.txt was deleted while the recent file2.log remained. Output ended with Cleanup completed successfully.	Y
log_rotate.sh	bash log_rotate.sh	Rotate log files while retaining exactly five previous versions	Script was executed for 7 runs. Final directory retained app.1, app.2, app.3, app.4, and app.5, with older versions discarded	Y
backup_auto.sh	bash backup_auto.sh	Create timestamped backup directory using rsync and record the operation in backup.log	Two timestamped backup directories were successfully created: backup_20260922_204714 and backup_20260922_204728	Y

SECTION 4: BASH AND PYTHON COMPARISON (analytical only, no Python code required)

Criterion	Bash	Python	Preferred for sysadmin work? (B/P)
Execution speed	Very fast for simple Linux commands and administration tasks	Fast, but may have more overhead for simple shell commands	B
Code readability / maintainability	Easy for short system administration scripts, but complex scripts can become difficult to read	Usually easier to organize and maintain for larger programs	P

System command integration	Excellent because Linux commands can be executed directly	System commands usually require modules such as subprocess or os	B
Error handling capability	Supports exit codes, set -euo pipefail, and traps	Provides structured exception handling using try and except	P
Available libraries	Primarily depends on installed Linux utilities and shell commands	Has a very large standard library and third-party package ecosystem	P

SECTION 5: CRON JOB VERIFICATION

Crontab entry (paste from crontab -l):	*/5 * * * * /bin/bash /home/jalandoon/lab1a_bash/script2_diskcheck.sh >> /home/jalandoon/lab1a_bash/cron_disk.log 2>&1
Cron log file path:	/home/jalandoon/lab1a_bash/cron_disk.log
Log entries found? (Y/N):	Y
Number of entries:	5 (atleast 1 entry was verified during testing)

SECTION 6: SCRIPT FILE LISTING (paste the output of ls -la ~/lab1a_bash/)

```
jalandoon@YANG:~/lab1a_bash$ ls -la ~/lab1a_bash/
total 68
drwxr-xr-x 8 jalandoon jalandoon 4096 Sep 22 21:41 .
drwxr-x--- 7 jalandoon jalandoon 4096 Sep 22 20:55 ..
-rw-r--r-- 1 jalandoon jalandoon  580 Sep 22 21:41 backup.log
drwxr-xr-x 2 jalandoon jalandoon 4096 Sep 22 20:46 backup_20260922_204714
drwxr-xr-x 2 jalandoon jalandoon 4096 Sep 22 20:46 backup_20260922_204728
drwxr-xr-x 2 jalandoon jalandoon 4096 Sep 22 21:40 backup_20260922_214132
drwxr-xr-x 2 jalandoon jalandoon 4096 Sep 22 21:40 backup_20260922_214140
-rwxr-xr-x 1 jalandoon jalandoon  590 Sep 22 21:41 backup_auto.sh
-rw-rw-r-- 1 jalandoon jalandoon  360 Sep 22 22:15 cron_disk.log
-rw-r--r-- 1 jalandoon jalandoon   17 Sep 22 21:37 error.log
-rwxr-xr-x 1 jalandoon jalandoon  807 Sep 22 21:39 log_rotate.sh
drwxr-xr-x 2 jalandoon jalandoon 4096 Sep 22 21:43 logs
-rwxr-xr-x 1 jalandoon jalandoon  490 Sep 22 21:27 script1_sysinfo.sh
-rwxr-xr-x 1 jalandoon jalandoon  416 Sep 22 21:29 script2_diskcheck.sh
-rwxr-xr-x 1 jalandoon jalandoon  489 Sep 22 21:31 script3_userreport.sh
-rwxr-xr-x 1 jalandoon jalandoon  632 Sep 22 21:38 script4_cleanup.sh
drwxr-xr-x 2 jalandoon jalandoon 4096 Sep 22 21:38 tmptest
jalandoon@YANG:~/lab1a_bash$
```

SECTION 7: GROUP REFLECTION (minimum 100 words)

In this laboratory activity, we learned how Bash scripting can be used to automate common Linux system administration tasks. We practiced storing command outputs in variables, using conditionals to evaluate disk usage, working with arrays and loops, creating functions, handling errors using strict mode and traps, rotating log files, and creating timestamped backups with rsync. We also learned how cron can automatically execute a script based on a schedule without manually running the command every time. One of the most important lessons we learned was the importance of testing scripts carefully before using them on actual system files, especially when commands such as find and -delete are involved. We therefore used a disposable test directory for the cleanup activity. Overall, the laboratory helped us understand how Bash scripts can make repetitive system administration tasks faster, more consistent, and easier to manage. It also improved our confidence in using the Linux terminal, troubleshooting command errors, checking log files, and verifying automated tasks.

SIGNATURES

Member 1	Member 2	Member 3	Member 4
JEREMIAH V. JALANDO-ON	KRISTY L. ESTILLORE	DAISY GRACE M. GENERALE	CYRINE GANLOY
Signature of Printed Name	Signature of Printed Name	Signature of Printed Name	Signature of Printed Name
Date	Date	Date	Date